
Security Review Report
NM-0986 - PROPHET MARKET LP VAULTS



NETHERMIND
SECURITY

(July 20, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
3	Summary of Issues	3
4	System Overview	4
4.1	System Architecture	4
4.2	Actors	4
4.3	Liquidity Provision and Exits	4
4.4	Fee Distribution and Tick Management	4
4.5	Vault Lifecycle and Emergency Procedures	4
5	Risk Rating Methodology	5
6	Issues	6
6.1	[Critical] Architectural contradiction between minting and reclaiming allows LPs to drain the Vault	6
6.2	[Critical] Fulfilling a pre-funded intent charges the LP twice and orphans the first deposit.	7
6.3	[Critical] LP providers cannot withdraw their deposits	7
6.4	[Critical] The LPVault contract is incompatible with ERC1155 tokens	8
6.5	[High] Missing unchecked blocks in fee growth arithmetic causes underflows, DOSing mints and locking LP funds	9
6.6	[Medium] Emergency withdrawal pays out recorded amounts without checking the vault can cover them	10
6.7	[Medium] The emergencyCancelAll(...) function permanently locks USDC for pending deposits by transitioning the vault to a terminal state without refunding unfulfilled intents	10
6.8	[Low] Admin transfer flow doesn't delete previous admin rights	11
6.9	[Low] The minimumFirstLiquidity check relies on activeLiquidity leading to potential DOS for LPs when market price enters empty tick ranges	11
6.10	[Low] Unbounded storage loop in _nextInitializedTick(...) allows attackers to DOS updateTick(...)	12
6.11	[Low] Unbounded loop over nextPositionId in emergencyCancelAll(...) leads to DOS due to running OOG	13
6.12	[Low] Upward tick search cannot report "no tick found" and reverts instead	13
6.13	[Info] Admin removing an operator, breaks the reclaimDeposit function	14
6.14	[Info] Lack of duplicate ID validation in mergePositions(...) allows artificial liquidity inflation and subsequent vault drain	14
6.15	[Info] Unused function _clearTickBitmapBit	14
6.16	[Info] Stale markets can be cancelled even if the Operator is live	15
6.17	[Info] USDC blacklist of a single LP leads to DOS of emergencyCancelAll(...) permanently locking all vault funds	15
7	Documentation Evaluation	16
8	Test Suite Evaluation	17
8.1	Tests Output	17
9	About Nethermind	26

1 Executive Summary

This document presents the results of the security review conducted by [Nethermind Security](#) for Prophet. The focus of the engagement has been the LPVaults contracts, a concentrated-liquidity and liquidity management system designed to operate alongside the ProphetCTFExchange.

The protocol allows liquidity providers (LPs) to supply USDC collateral into specific price ranges for prediction markets to earn trading fee yield. Users sign EIP712 mint intents off-chain, which a trusted Operator submits and settles on-chain through the per-market LPVault contract. The Operator is also responsible for synchronizing the vault's current price tick with the off-chain Central Limit Order Book (CLOB), distributing collected fee revenue across active liquidity, and consolidating user positions via merge operations to optimize gas.

To facilitate per-market accounting, the system utilizes an LPVaultFactory to deploy EIP1167 minimal proxy clones for each prediction market. These vaults hold USDC and ERC1155 outcome tokens, managing the full lifecycle from active trading to an Oracle-triggered wind-down once the market resolves. Because the Operator is trusted to process intents and notify fees, the system implements LP exit flows in case the Operator goes offline. These include a 24 hour timelocked deposit reclaim for unfulfilled mint intents, and a 7 day (emergencyCancelAll) function that force-closes all positions and returns funds to users if the Operator stops responding.

The audit comprises 790 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report seventeen points of attention, where four are classified as Critical, one is classified as High, two are classified as Medium, four are classified as Low, and five are classified as Informational severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

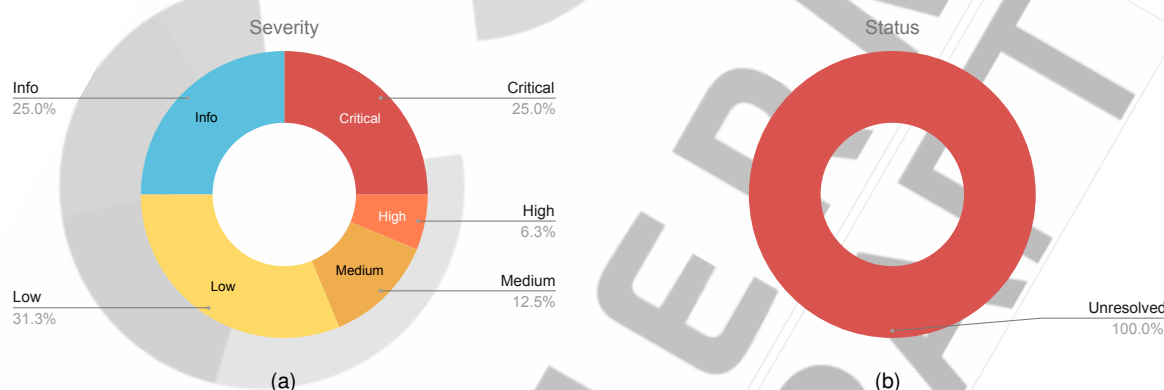


Fig. 1: Distribution of issues: Critical (4), High (1), Medium (2), Low (5), Undetermined (0), Informational (5), Best Practices (0). Distribution of status: Fixed (0), Acknowledged (0), Mitigated (0), Unresolved (17)

Summary of the Audit

Audit Type	Security Review
Initial Report	July 20, 2026
Final Report	-
Initial Commit	d47b72d
Final Commit	-
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	/src/LPVaultFactory.sol	155	133	85.8%	67	355
2	/src/LPVault.sol	635	495	78.0%	235	1365
	Total	790	628	79.5%	302	1720

3 Summary of Issues

	Finding	Severity	Update
1	Architectural contradiction between minting and reclaiming allows LPs to drain the Vault	Critical	Not Fixed
2	Fulfilling a pre-funded intent charges the LP twice and orphans the first deposit.	Critical	Not Fixed
3	LP providers cannot withdraw their deposits	Critical	Not Fixed
4	The LPVault contract is incompatible with ERC1155 tokens	Critical	Not Fixed
5	Missing unchecked blocks in fee growth arithmetic causes underflows, DOSing mints and locking LP funds	High	Not Fixed
6	Emergency withdrawal pays out recorded amounts without checking the vault can cover them	Medium	Not Fixed
7	The emergencyCancelAll(...) function permanently locks USDC for pending deposits by transitioning the vault to a terminal state without refunding unfulfilled intents	Medium	Not Fixed
8	Admin transfer flow doesn't delete previous admin rights	Low	Not Fixed
9	The minimumFirstLiquidity check relies on activeLiquidity leading to potential DOS for LPs when market price enters empty tick ranges	Low	Not Fixed
10	Unbounded storage loop in _nextInitializedTick(...) allows attackers to DOS updateTick(...)	Low	Not Fixed
11	Unbounded loop over nextPositionId in emergencyCancelAll(...) leads to DOS due to running OOG	Low	Not Fixed
12	Upward tick search cannot report "no tick found" and reverts instead	Low	Not Fixed
13	Admin removing an operator, breaks the reclaimDeposit function	Info	Not Fixed
14	Lack of duplicate ID validation in mergePositions(...) allows artificial liquidity inflation and subsequent vault drain	Info	Not Fixed
15	Unused function _clearTickBitmapBit	Info	Not Fixed
16	Stale markets can be cancelled even if the Operator is live	Info	Not Fixed
17	USDC blacklist of a single LP leads to DOS of emergencyCancelAll(...) permanently locking all vault funds	Info	Not Fixed

4 System Overview

The LP Vault system is a liquid staking and concentrated liquidity protocol designed to operate alongside the ProphetCTFExchange. It facilitates the creation of per-market vaults that manage USDC collateral and ERC1155 Conditional Tokens. The architecture employs a centralized factory to deploy minimal proxy clones for individual prediction markets, allowing Liquidity Providers (LPs) to supply concentrated liquidity into specific price ranges.

4.1 System Architecture

- **LPVaultFactory:** The central deployment and configuration hub. It uses the EIP1167 minimal proxy pattern to deploy new LPVault instances for specific markets. It also maintains the global registry of administrative roles (Admins, Operators, Oracles) and manages a 7 day timelocked upgrade path for the vault implementation.
- **LPVault:** The per-market instance responsible for holding USDC and outcome tokens. It manages individual LP positions, tracks active liquidity across price ticks, and handles the distribution of trading fees.

4.2 Actors

- **Admin:** The highest-level governance authority. Admins manage the role registry on the factory contract, assigning or revoking Operator and Oracle privileges. They are also responsible for scheduling implementation upgrades and can trigger a `pauseTrading()` circuit breaker on individual vaults.
- **Oracle:** The lifecycle manager. The Oracle creates new vaults via the factory and is responsible for calling `startWindDown()` to transition a vault into its exit-only phase once the underlying market resolves. To enforce separation of duties, an Oracle cannot simultaneously act as an Operator.
- **Operator:** The primary operational driver. Operators process off-chain EIP712 signed intents from LPs to mint on-chain positions. They update the vault's price tick to match the off-chain Central Limit Order Book (CLOB) and distribute collected fee revenue.
- **Liquidity Providers (LPs):** Users who deposit USDC to earn yield. They sign off-chain intents to define their desired concentrated liquidity bounds and deposit amounts. Once positions are minted, LPs can claim accrued fees or trigger escape hatches.

4.3 Liquidity Provision and Exits

Liquidity provision utilizes an intent-based architecture to minimize gas costs and slippage for end users. LPs construct a `MintIntent` detailing their desired lower and upper ticks and sign it via EIP712. The Operator submits this signature on-chain via the `mintPositionFor(...)` function, pulling the LP's USDC into the vault and updating the active liquidity if the position is within the current price range.

LPs have multiple avenues for exiting or managing their capital:

- **Fee Collection:** LPs can call `collect(...)` at any time to withdraw accrued USDC trading fees without closing their position.
- **Deposit Reclaim:** If an LP's off-chain deposit is not serviced by the Operator, they can invoke `reclaimDeposit(...)`. This is a two-step exit flow governed by a 24-hour timelock, allowing the LP to recover their initial USDC if the `MintIntent` remains unfulfilled.
- **Position Merging:** The Operator can call `mergePositions(...)` to consolidate multiple positions owned by the same LP within the exact same tick range, rolling up uncollected fees and preserving total liquidity to optimize future gas costs.

4.4 Fee Distribution and Tick Management

The vault dynamically tracks price movements and distributes fees across active liquidity ranges.

- **Tick Updates:** As the market price shifts, the Operator calls `updateTick(...)` to synchronize the vault's internal state. This function crosses initialized ticks, flipping fee accumulators and adding or removing net liquidity from the active pool to ensure only in-range positions earn yield.
- **Fee Notification:** Trading fees swept from the exchange are distributed via `notifyFees(...)`. The Operator deposits the USDC and increments the global fee accumulator proportionally across all currently active liquidity.

4.5 Vault Lifecycle and Emergency Procedures

Each vault has the following lifecycle: Active, WindDown, and Cancelled.

- **Wind-Down:** Once the underlying market concludes, the Oracle triggers the WindDown phase. This halts the creation of new positions but allows LPs to continue collecting fees and reclaiming deposits.
- **Emergency Cancel:** To protect against the Operator going offline, the system features a switch. If no fees are notified or ticks updated for 7 days, any active position holder can call `emergencyCancelAll(...)`. This force-closes all positions, calculates principal and uncollected fees, zeros the vault's state, and distributes the remaining USDC to users, transitioning the vault to the Cancelled phase.

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Critical] Architectural contradiction between minting and reclaiming allows LPs to drain the Vault

File(s): `src/LPVault.sol`

Description: The LPVault contract allows liquidity providers to mint concentrated-liquidity positions using the `mintPositionFor(...)` function. This function is gated to operators who submit signed EIP-712 intents on behalf of the users. To protect users from unresponsive operators, the contract implements a `reclaimDeposit(...)` function, allowing users to reclaim unfulfilled deposits after a 24-hour timelock.

There is a logic flaw between how these two functions handle user funds. The `mintPositionFor(...)` function uses a pull-based model, actively pulling the `usdcAmount` from the user's wallet via the `_safeTransferFrom(...)` function. If an intent is not executed, the user's tokens remain in their own wallet.

```

1  function mintPositionFor(...)
2      external
3      onlyOperator
4      whenNotPaused
5      nonReentrant
6      returns (uint256 positionId)
7  {
8      // ...
9      // @audit-issue The function pulls funds from the user's wallet.
10     _safeTransferFrom(usdc, lp, address(this), usdcAmount);
11     // ...
12 }
13

```

Conversely, the `reclaimDeposit(...)` function operates on a "deposit-then-credit" assumption. When executed, it directly transfers the `usdcAmount` from the vault to the user via the `_safeTransfer(...)` function, assuming that the unfulfilled intent's funds were already deposited and held by the vault.

```

1  function reclaimDeposit(...) external nonReentrant {
2      // ...
3      usedIntents[intentId] = true;
4
5      // @audit-issue The vault pushes funds to the user without prior deposit verification.
6      _safeTransfer(usdc, lp, usdcAmount);
7      // ...
8  }
9

```

An attacker can exploit this discrepancy to drain the vault's reserves by generating a valid mint intent withhold providing an ERC20 allowance to the vault. This prevents the operator from successfully executing the `mintPositionFor(...)` function because the USDC transfer will revert. Once the 24 hour timelock passes, the attacker can call the `reclaimDeposit(...)` function to receive the `usdcAmount` from the vault, effectively stealing funds from other users' liquidity without depositing any tokens.

Recommendation(s): Consider revisiting the logic of the two flows.

Status: Unresolved

Update from the client:

6.2 [Critical] Fulfilling a pre-funded intent charges the LP twice and orphans the first deposit.

File(s): [src/LPVault.sol](#),

Description: The specification documents a **deposit-then-credit** flow in which the LP wires `usdcAmount` to the vault `_before_` the Operator fulfills the signed intent (FEAT-T7AF "deposit-then-credit flow (USDC pre-sent to vault)"; UC-JAIK preconditions "wired USDC to the vault"; SC-JAIL "wired `usdcAmount` to the vault"). The entire `reclaimDeposit` escape hatch is built on this assumption — it refunds `usdcAmount` straight out of the vault balance for an intent that was never minted, which is only sound if the LP actually pre-sent that USDC. The two paths are made mutually exclusive through the shared `usedIntents` flag.

`mintPositionFor`, however, does not consume or account for the pre-sent deposit. It performs a second, independent pull from the LP:

```
1  _safeTransferFrom(usdc, lp, address(this), usdcAmount);
```

There is no balance reconciliation anywhere in the function — it never reads the vault's USDC balance or nets out the amount already parked against the intent. So when an LP follows the documented flow:

- If the LP does not keep a `_second_ usdcAmount` and allowance in their wallet, fulfillment reverts even though the intended deposit is already sitting in the vault;
- If the second pull succeeds, the vault receives two `usdcAmount` deposits while only one position is created, with principal derived from a single `usdcAmount` (liquidity computed at [LPVault.sol:631](#));

The intent is then permanently marked used, so `reclaimDeposit` can no longer recover the first transfer. The orphaned pre-sent amount has no per-intent accounting and is not part of any position, so it is also excluded from the emergency payout, whose principal is re-constructed purely from position liquidity and tick width ([LPVault.sol:552-554](#)). The extra deposit is left unowned in the vault — silently subsidizing other LPs' claims or trapped permanently.

Recommendation(s): Consider picking one deposit model where either (a) have `mintPositionFor` credit the already-deposited `usdcAmount` instead of pulling again — verifying on-chain that the vault balance covers the intent before creating the position — or (b) drop the pre-send model and rework `reclaimDeposit` so it can only refund funds the contract itself observed being escrowed against that specific intent.

Status: Unresolved

Update from the client:

6.3 [Critical] LP providers cannot withdraw their deposits

File(s): [src/LPVault.sol](#)

Description: Once an LP's position is minted, the LP has **no way to withdraw their principal**. There is no function to close a position and return its deposit, and none of the existing functions give principal back to an LP on demand:

- `collect` pays out accrued fees only — it never returns principal;
- `reclaimDeposit` only applies to intents that were never minted; it cannot touch an active position, and it also requires an Operator co-signature;
- `emergencyCancelAll` is the only path that returns principal, but it can only be called after 7 days of Operator inactivity — and any `updateTick` or `notifyFees` call resets that timer, so an active Operator can keep it permanently out of reach;

The result is that an LP's deposited funds are locked in the vault for as long as the Operator keeps operating. LPs cannot exit on their own; every principal exit depends on the Operator. A hostile or simply uncooperative Operator can trap LP capital indefinitely.

Recommendation(s): Consider adding a self-service withdrawal path that lets a position owner close their position and receive their principal (plus any accrued fees) at any time, independent of the Operator, with the corresponding tick-state cleanup.

Status: Unresolved

Update from the client:

6.4 [Critical] The LPVault contract is incompatible with ERC1155 tokens

File(s): `src/LPVault.sol`, `contracts/token/ERC1155/ERC1155.sol`

Description: The LPVault contract acts as a liquidity provider and it is meant to hold both USDC and ERC1155 Gnosis Conditional Tokens for the market. This is confirmed by the natspec and the fact that the `initialize` function provides approval to the exchange to manage these ERC1155 tokens on behalf of the Vault.

```
1  /// @title LPVault
2  /// @notice Per-market vault holding USDC and ERC-1155 outcome tokens.
3
4  //..
5      function initialize(
6          bytes32 marketId_,
7          address usdc_,
8          address exchange_,
9          address conditionalTokens_,
10         int24 tickSpacing_,
11         address factory_,
12         uint128 minimumFirstLiquidity_,
13         uint256 version_
14     ) external initializer {
15 //..
16
17 //@audit
18     IERC1155(conditionalTokens_).setApprovalForAll(exchange_, true);
19 }
20
```

During normal trading operations, the exchange routes trades against the vault, pulling its USDC and sending back purchased outcome tokens.

However, the ERC1155 token standard (that can be seen [here](#)) requires that any smart contract receiving tokens via the `safeTransferFrom(...)` or `safeBatchTransferFrom(...)` functions must implement the `IERC1155Receiver` interface and return the appropriate magic value. The LPVault contract doesn't implement the `onERC1155Received(...)` and `onERC1155BatchReceived(...)` functions.

Because these functions are missing, the Conditional Tokens contract will revert transfers. This creates a permanent DOS because the vault is incapable of successfully completing any trade that requires it to receive outcome tokens. Every transaction where the exchange attempts to send Conditional Tokens to the vault will revert, making the vault useless.

Recommendation(s): Consider implementing the missing functions.

Status: Unresolved

Update from the client:

6.5 [High] Missing unchecked blocks in fee growth arithmetic causes underflows, DOSing mints and locking LP funds

File(s): [src/LPVault.sol](#)

Description: The `_computeFeeGrowthInside(...)` function from the `LPVault` contract calculates accumulated fees inside a specific tick range. This calculation is vital for the lazy initialization of ticks, where a newly initialized tick assumes all historic fees occurred outside its bounds relative to the `currentTick`.

When a liquidity provider mints a position that shares one previously initialized tick and one new tick, the sum of `feeGrowthBelow` and `feeGrowthAbove` can mathematically exceed the `feeGrowthGlobalX128` variable. In Uniswap V3, this is handled by allowing modular arithmetic to correctly represent the delta as a wrapped negative value, but Uniswap V3 is compiled in a version of Solidity that's lower than 0.8. However, the `LPVault` contract compiles with Solidity 0.8.20 and doesn't wrap the math in an unchecked block.

```

1  function _computeFeeGrowthInside(...) internal view returns (uint256) {
2      // ...
3      uint256 feeGrowthAbove;
4      if (currentTick < tickUpper) {
5          feeGrowthAbove = ticks[tickUpper].feeGrowthOutsideX128;
6      } else {
7          feeGrowthAbove = feeGrowthGlobalX128 - ticks[tickUpper].feeGrowthOutsideX128;
8      }
9
10     // @audit-issue Subtraction without an unchecked block may lead to underflow.
11     return feeGrowthGlobalX128 - feeGrowthBelow - feeGrowthAbove;
12 }
13

```

Furthermore, even if the `_computeFeeGrowthInside(...)` function were patched, the fee delta calculations in the `collect(...)`, `emergencyCancelAll(...)`, and `mergePositions(...)` functions also lack unchecked blocks.

```

1  function collect(...) external nonReentrant {
2      // ...
3      uint256 feeGrowthInsideX128 = _computeFeeGrowthInside(p.tickLower, p.tickUpper);
4
5      // @audit-issue Missing unchecked block when calculating the fee delta.
6      uint256 owed = uint256(p.liquidity) * (feeGrowthInsideX128 - p.feeGrowthInsideLastX128) / Q128;
7      // ...
8  }
9

```

As global fees grow, the wrapped negative value will naturally cross back over the 256-bit maximum boundary into positive territory. Subtracting the large wrapped snapshot from the new small positive value will again trigger an underflow. This breaks liquidity provision for those ranges. Attackers can intentionally mint small positions to "mine" ticks and purposefully brick standard ranges. Additionally, any liquidity provider whose position relies on the wrapped fee arithmetic will have their funds stuck, as the `collect(...)` and `emergencyCancelAll(...)` functions will always revert with a `Panic(0x11)` error.

Recommendation(s): Consider wrapping the fee growth subtractions in unchecked blocks to allow modular arithmetic. Review the `_computeFeeGrowthInside(...)` function and all fee delta calculations across the contract to ensure they can properly handle wrapped values without triggering reverts on underflow.

Status: Unresolved

Update from the client:

6.6 [Medium] Emergency withdrawal pays out recorded amounts without checking the vault can cover them

File(s): [src/LPVault.sol](#)

Description: `emergencyCancelAll` computes each owner's payout entirely from the vault's internal records — a principal figure reconstructed from the position's liquidity and tick width, plus accrued fees — and then transfers those amounts out in a loop. At no point does it check that the vault actually holds enough USDC to cover the total. It simply assumes the recorded entitlements and the real balance agree.

They frequently will not agree. The vault's balance is moved independently by the exchange (via the standing approval), and its recorded entitlements can be inflated by other issues in this report — fees credited without funding, a reclaim that paid out un-deposited funds, or a position whose liquidity was doubled by the merge bug. Whenever the sum of payouts exceeds the vault's real balance, the loop pays the earliest owners and then reverts when the balance runs dry. Because every state change (including marking the vault cancelled) is rolled back on that revert, the function does not partially settle — it simply **fails every time it is called** while the vault is short.

The consequence is that the one recovery path meant to return everyone's funds becomes permanently un-callable exactly when the vault is under-funded — which is the situation in which LPs most need it. Combined with the absence of any self-service withdrawal and the ways an Operator can hold off the emergency timer or exhaust its gas, LP principal can be left with no working exit at all.

Recommendation(s): Consider implementing a logic where before paying out, the amounts owed is totalled and compare against the vault's actual balance; if the balance is short, distribute what exists pro-rata across claimants rather than paying the first few in full and reverting for the rest then tie this to the balance-reconciliation safeguard.

Status: Unresolved

Update from the client:

6.7 [Medium] The `emergencyCancelAll(...)` function permanently locks USDC for pending deposits by transitioning the vault to a terminal state without refunding unfulfilled intents

File(s): [src/LPVault.sol](#)

Description: The `LPVault` contract implements a deposit-then-credit architecture where liquidity providers deposit tokens and sign a mint intent. If the operator fails to execute this intent via the `mintPositionFor(...)` function, the user can recover their funds using the `reclaimDeposit(...)` function. The contract also features a global exit flow via the `emergencyCancelAll(...)` function for when the vault winds down or the operator goes offline.

However, when the `emergencyCancelAll(...)` function is called while there are still pending, unminted deposits, it iterates exclusively through active, minted positions (from 0 to the `nextPositionId`) to distribute their principal and fees. Because pending deposits have not yet been assigned a position ID and their liquidity hasn't been added into the positions mapping, they are bypassed by this payout loop. The function then transitions the vault to the terminal cancelled phase.

```

1  function emergencyCancelAll() external nonReentrant {
2      // ...
3      uint256 count = nextPositionId;
4      // ...
5      for (uint256 i = 0; i < count; i++) {
6          // ...
7          // @audit-issue The vault transitions to a terminal state without refunding pending deposits.
8          phase = 3;
9          // ...
10     }

```

Furthermore, the `reclaimDeposit(...)` function explicitly reverts if the vault is in the cancelled phase (`phase == 3`).

```

1  function reclaimDeposit(...) external nonReentrant {
2      // ...
3      // @audit The escape hatch reverts if the vault is cancelled, locking pending deposits.
4      if (phase == 3) revert VaultCancelled();
5      // ...
6  }

```

Consequently, if the `EMERGENCY_CANCEL_TIMELOCK` elapses and an active position holder calls the `emergencyCancelAll(...)` function before a pending user can complete their 24-hour reclaim process, the pending deposits are neither refunded by the emergency sweep nor recoverable via `reclaimDeposit`. This permanently locks the users' tokens in the dead vault.

Recommendation(s): Consider adding guardrails so the vault can not be closed while pending deposits exist or, refund the pending deposits before closing the vault.

Status: Unresolved

Update from the client:

6.8 [Low] Admin transfer flow doesn't delete previous admin rights

File(s): `src/LPVaultFactory.sol`

Description: The LPVaultFactory contract implements a two step admin transfer flow. However, the old admin is never removed from the admins mapping. This means that the onlyAdmin modifier check will still pass for the old admin address as well as the admins checks in the LPVault contract.

```

1  function acceptAdmin() external {
2      if (msg.sender != pendingAdmin) revert NotPendingAdmin();
3      // @audit-issue the old admin is not removed from the `admins` mapping
4      admins[msg.sender] = 1;
5      adminCount += 1;
6      pendingAdmin = address(0);
7
8      // ..
9  }
```

Recommendation(s): Consider removing the old admin's rights by setting the corresponding value in the admins mapping back to zero.

Status: Unresolved

Update from the client:

6.9 [Low] The minimumFirstLiquidity check relies on activeLiquidity leading to potential DOS for LPs when market price enters empty tick ranges

File(s): `src/LPVault.sol`

Description: The LPVault contract allows liquidity providers to mint concentrated-liquidity positions using the mintPositionFor(...) function. To prevent dust-based initialization attacks, the protocol enforces a minimum liquidity floor defined by the minimumFirstLiquidity variable for the very first deposit in the vault.

However, the current logic uses the activeLiquidity state variable as a way to determine if the vault is uninitialized. In a concentrated liquidity model, activeLiquidity tracks the liquidity that is _currently in range_. Whenever the market price (tracked by the currentTick) moves into a tick range where no existing positions overlap, the activeLiquidity drops to 0 again.

The core issue is that this check, which is meant to run only once upon initializing the vault, will actually trigger every single time the ticks move into an area where there is no liquidity added yet.

```

1  function mintPositionFor(...) external onlyOperator whenNotPaused nonReentrant returns (uint256 positionId) {
2      // ...
3      uint128 liquidity = _toUint128(usdcAmount * LIQUIDITY_PRECISION / rangeWidth);
4      // @audit-issue This check triggers every time `activeLiquidity` drops to 0, not just on initialization.
5      if (activeLiquidity == 0 && liquidity < minimumFirstLiquidity) {
6          revert BelowMinimumFirstLiquidity(); // ...
7      }
8  }
```

As it stands, the protocol's assumption that this is a "First-mint floor check (FR-RFS7 from FEAT-REPZ)" is flawed, because this can trigger even after the Vault was initialized and there are active LP in the market. Depending on the size of minimumFirstLiquidity it could lead to DOS for smaller LPs when they try to mint new positions, until a user adds enough liquidity that satisfies the check.

Recommendation(s): Consider changing the condition to check if the position being minted is the first one in the vault's history. You can use the nextPositionId counter to enforce the floor exactly once upon vault initialization.

Status: Unresolved

Update from the client:

6.10 [Low] Unbounded storage loop in `_nextInitializedTick(...)` allows attackers to DOS `updateTick(...)`

File(s): `src/LPVault.sol`

Description: The `updateTick(...)` function in the LPVault contract is used to synchronize the vault's price by crossing initialized ticks. It achieves this by calling the `_nextInitializedTick(...)` function within a while loop to locate the next initialized tick.

However, the `_nextInitializedTick(...)` function does not accept the operator's target `newTick` as an argument. Instead, it executes an unbounded for loop across the `tickBitmap` storage space, searching until it finds a set bit or hits the `type(int16).max` or `type(int16).min` boundaries.

An attacker can exploit this design by signing a valid `MintIntent` with a minimal amount of tokens at an extreme tick value (e.g., a `tickLower` of 8388600), which translates to a `bitmap wordPos` near 32767. When a normal market movement occurs and the operator calls the `updateTick(...)` function to adjust the price, the contract will scan the empty `bitmap` starting from the current `wordPos`.

```

1  function updateTick(int24 newTick) external onlyOperator whenNotPaused nonReentrant {
2      // ...
3      if (movingRight) {
4          // ...
5          while (tick < newTick) {
6              // @audit The `newTick` target is not passed to bound the search.
7              (int24 next, bool found) = _nextInitializedTick(tick, true);
8              if (!found || next > newTick) break;
9              //...
10 }

```

If the required gap exists, the `_nextInitializedTick(...)` function loops through storage until it finds the attacker's set bit.

```

1  function _nextInitializedTick(int24 tick, bool searchRight) internal view
2      returns (int24 next, bool found)
3  {
4      if (searchRight) {
5          // ...
6          wordPos++;
7          // @audit-issue Unbounded loop searches up to `type(int16).max` (32767).
8          for (; wordPos <= type(int16).max; wordPos++) {
9              word = tickBitmap[wordPos];
10             if (word != 0) {
11                 // ...
12                 return (0, false);
13             } else {
14                 // ...
15                 wordPos--;
16                 // @audit-issue Unbounded loop searches down to `type(int16).min` (-32768).
17                 for (; wordPos >= type(int16).min; wordPos--) {
18                     word = tickBitmap[wordPos];
19                     if (word != 0) {
20                         // ...
21                         return (0, false);
22                     }
23 }

```

This loop requires over 32,000 cold storage reads, reverting due to the block gas limit. This causes a denial of service of the `updateTick(...)` function, which will break the vault's core economic invariants. The `activeLiquidity` and fee growth boundaries become frozen. The `notifyFees(...)` function will continue to succeed but will distribute yield based on stale data.

This attack has certain preconditions: it requires a big, uninterrupted gap of uninitialized ticks between the current price and the attacker's distant tick. If organic liquidity is present along the way, the loop will find other initialized ticks along the way and it `_SHOULD_`, safely break out of the loop. However, if a gap exists, a malicious user can DOS the protocol.

Recommendation(s): Consider refactoring the `_nextInitializedTick(...)` function to limit the search depth to the operator's intended target. Pass the `newTick` value into the search function and terminate the loop if it reaches the target's `wordPos` without finding a set bit. This will bound gas execution per call.

Status: Unresolved

Update from the client:

6.11 [Low] Unbounded loop over nextPositionId in emergencyCancelAll(...) leads to DOS due to running OOG

File(s): [src/LPVault.sol](#)

Description: The emergencyCancelAll(...) function in the LPVault contract serves as the sole mechanism for liquidity providers to recover their deposited principal when the vault winds down or the operator goes offline. It operates by looping from 0 to the nextPositionId to compute and process payouts for every position ever minted.

Because the nextPositionId state variable monotonically increases on every mint and there is no mechanism to delete positions from storage, popular vaults with high mint volumes might end up having a very big nextPositionId. Iterating over thousands of positions, reading storage for tick data, computing fees, and executing external transfers per iteration will eventually exceed network block gas limits.

```

1  function emergencyCancelAll() external nonReentrant {
2      // ...
3      // @audit-issue The `nextPositionId` counter only increases, causing an unbounded loop.
4      uint256 count = nextPositionId;
5      // ...
6      // @audit-issue reading all positions and writing in storage here
7      for (uint256 i = 0; i < count; i++) {
8          Position storage p = positions[i];
9          if (p.liquidity == 0) continue;
10         // ...
11         // @audit-issue reading all positions again and doing a transfer
12         for (uint256 i = 0; i < count; i++) {
13             if (payouts[i] > 0) {
14                 _safeTransfer(usdc, owners[i], payouts[i]);
15             }
16         }
17     }

```

If there are too many positions created for a given vault, the emergencyCancelAll(...) function will eventually revert with an out of gas error. This leads to permanent DOS of the vault's only exit path, permanently locking all deposited funds.

Recommendation(s): Consider revisiting the exit flow. For example, the global emergencyCancelAll(...) function could be redesigned to only freeze the vault and record the final global state. Then, liquidity providers could call a withdrawal function to process their own positions or, consider adding a batching variant where smaller chunks of LP withdrawals can be processed.

Status: Unresolved

Update from the client:

6.12 [Low] Upward tick search cannot report "no tick found" and reverts instead

File(s): [src/LPVault.sol](#)

Description: _nextInitializedTick must be able to report two outcomes: the next initialized tick in the requested direction, or that there is none (found = false). Its two directions are not symmetric.

The downward search decrements the bitmap word index and has an explicit guard that stops when it reaches the lowest word (LPVault.sol:1197), so it exits cleanly and returns found = false.

The upward search has no matching guard. Its loop is `for (; wordPos <= type(int16).max; wordPos++)`, and wordPos is an int16. To exit the loop normally, wordPos would have to exceed int16 max, but the incrementing wordPos++ overflows int16 and reverts (Solidity 0.8 checked arithmetic) before that can happen. The `return (0, false)` that follows the loop is therefore unreachable: the upward search can never report "no tick found" — it reverts with an arithmetic panic instead. The pre-loop increment/decrement that both branches perform before entering the loop are likewise unguarded at the extreme word.

This is a correctness defect distinct from the gas exhaustion in finding 15. It occurs independently of gas: when a position sits near the top of the tick range, the upward search starts near the boundary and overflows after only a few steps, with no large scan involved. In both cases the effect compounds finding 15 — moving the price above all liquidity reverts — but here the cause is a missing boundary guard rather than the size of the scan.

Recommendation(s): Consider adding the symmetric boundary guard to the upward search (`if (wordPos == type(int16).max) break;`) and guard the pre-loop increment/decrement against the extreme word, so the function can return found = false at both ends.

Status: Unresolved

Update from the client:

6.13 [Info] Admin removing an operator, breaks the reclaimDeposit function

File(s): `src/LPVault.sol`, `src/LPVaultFactory.sol`

Description: The `reclaimDeposit(...)` function in the `LPVault` contract is designed to refund liquidity providers if an operator fails to mint their position. This exit path requires a co-signature from the Operator to prove that the deposit occurred.

During execution, the `_verifyOperatorSignature(...)` function calls the factory contract to ensure the operator is active.

```

1 function _verifyOperatorSignature(...) internal view {
2     // ...
3     address signer = ecrecover(digest, v, r, s);
4     // @audit-issue The validation relies on a real-time role check from the factory.
5     if (signer == address(0) || ILPVaultFactory(factory).operators(signer) != 1) revert InvalidSignature();
6 }

```

If an operator signed the mint intent, but it is removed by the protocol in the meantime, this invalidates all their previously issued co-signatures. Consequently, any user with a pending deposit who attempts to call the `reclaimDeposit(...)` function will face an `InvalidSignature` revert. If the admin removes the operator during the 24-hour timelock between Phase 1 and Phase 2 of the reclaim process, users who already initiated the reclaim process are blocked from completing it. This will lock user funds in the vault.

Recommendation(s): Consider checking for pending deposits/reclaims before removing an Operator.

Status: Unresolved

Update from the client:

6.14 [Info] Lack of duplicate ID validation in mergePositions(...) allows artificial liquidity inflation and subsequent vault drain

File(s): `src/LPVault.sol`

Description: The `mergePositions(...)` function from the `LPVault` contract is used by operators to combine multiple liquidity provider positions into a single survivor position to save on gas costs. However, the function does not validate whether the provided `positionIds` array contains duplicate elements.

If an operator passes an array with duplicate IDs (e.g., `[1, 1, 1]`), the function will assign the same storage slot to both the survivor storage pointer and the consumed storage pointer. During the loop iteration, the duplicate liquidity is accumulated into the `totalLiquidity` memory variable, and the consumed position in storage is zeroed out. Finally, the function overwrites the storage slot with the artificially inflated `totalLiquidity` sum. This effectively multiplies the position's liquidity without actually increasing the underlying `activeLiquidity` or `tickLiquidityGross` variables. Because fee distributions and `emergencyCancelAll(...)` payouts scale proportionally with the user's recorded liquidity, the owner of the inflated position will be able to withdraw a disproportionate share of the vault's tokens, draining other users' principal and accrued fees.

```

1 function mergePositions(uint256[] calldata positionIds) external onlyOperator /* ... */ {
2     // ...
3     for (uint256 i = 1; i < positionIds.length; i++) {
4         // @audit-issue No duplicate validation; consumed and survivor can point to the same positionId.
5         Position storage consumed = positions[positionIds[i]];
6         // ...
7         totalLiquidity += consumed.liquidity;
8         // ...
9         consumed.liquidity = 0;
10        // ...
11        // @audit This overwrites the previously zeroed out position with the inflated liquidity.
12        survivor.liquidity = totalLiquidity;
13        // ...

```

Recommendation(s): Consider adding validation inside the `mergePositions(...)` function to ensure that all elements within the `positionIds` array are unique.

Status: Unresolved

Update from the client:

6.15 [Info] Unused function _clearTickBitmapBit

File(s): `src/LPVault.sol`

Description: The `_clearTickBitmapBit` function of the `LPVault` contract is unused.

Recommendation(s): Consider removing the dead code.

Status: Unresolved

Update from the client:

6.16 [Info] Stale markets can be cancelled even if the Operator is live

File(s): [src/LPVault.sol](#)

Description: The LPVault contract implements an `emergencyCancelAll(...)` exit flow that is protected by a 7-day operator silence timelock (EMERGENCY_CANCEL_TIMELOCK). This mechanism relies on the `lastOperatorActivityTimestamp` state variable to determine if an operator has gone offline.

However, this timer is only updated when the operator calls the `notifyFees(...)` or `updateTick(...)` functions. If a market experiences a prolonged period of inactivity or low trading volume, the operator may not update the heartbeat in a timely manner. The `updateTick(...)` function reverts if the tick has not changed, preventing the timer from being refreshed. Similarly, the `notifyFees(...)` function reverts if there are no new fees to distribute, meaning a low-volume market will also prevent the operator from registering activity.

```
1 function notifyFees(uint256 amount) external onlyOperator whenNotPaused {
2     // ...
3     // @audit-issue Reverts on zero fees, preventing a heartbeat update.
4     if (amount == 0) revert ZeroAmount();
5     // ...
6 }
7 function updateTick(int24 newTick) external onlyOperator whenNotPaused nonReentrant {
8     // ...
9     int24 oldTick = currentTick;
10    // @audit-issue Reverts if the price is stable, preventing a heartbeat update.
11    if (newTick == oldTick) revert SameTick();
12    // ...
```

Even if the operator is healthy and actively processing user deposits via the `mintPositionFor(...)` function, the timer might expire because minting does not update the timestamp. Once 7 days pass, any user can trigger the `emergencyCancelAll(...)` function. This transitions the vault to the cancelled phase, force-closing all positions and ending the market.

Recommendation(s): Consider adding a function to update the `lastOperatorActivityTimestamp` variable outside of the two flows mentioned above in case the Operator is online, but there are no new fees and the price in the market hasn't moved enough. Alternatively, the `lastOperatorActivityTimestamp` could also be updated inside the merge and mintPosition flows which signals that the Operator is live.
Status: Unresolved

Update from the client:

6.17 [Info] USDC blacklist of a single LP leads to DOS of emergencyCancelAll(...) permanently locking all vault funds

File(s): [src/LPVault.sol](#)

Description: The `emergencyCancelAll(...)` function from the LPVault contract serves as the escape mechanism for liquidity providers to recover their deposited principal and fees when the operator goes offline or the vault enters a terminal state.

However, the function implements a push-payment model. It iterates through all positions in the vault and calls the `_safeTransfer(...)` function for each owner to distribute their respective payouts. The USDC token contract enforces a blacklist mechanism for compliance purposes. If a single position owner is blacklisted by the USDC contract, their transfer will revert. Because these transfers are executed sequentially within a single global loop, a single revert will bubble up and fail the entire `emergencyCancelAll(...)` transaction.

```
1 function emergencyCancelAll() external nonReentrant {
2     // ...
3     uint256 count = nextPositionId;
4     // ...
5     for (uint256 i = 0; i < count; i++) {
6         // ...
7         // ...
8         for (uint256 i = 0; i < count; i++) {
9             if (payouts[i] > 0) {
10                // @audit-issue Push-payment loop. A single revert fails the entire transaction.
11                _safeTransfer(usdc, owners[i], payouts[i]);
12            } // ...
```

Because there is no individual liquidity removal function available to users, this completely breaks the global escape mechanism. A single blacklisted address effectively traps the principal and accumulated fees of all legitimate liquidity providers within the vault permanently.

Recommendation(s): Consider wrapping the `_safeTransfer` call in a try-catch block and if the transfer fails, funds can be transferred to an Escrow contract/wallet that the protocol owns. Alternatively, consider skipping the recipient, but this will leave the USDC in the Vault.

Status: Unresolved

Update from the client:

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Prophet's contracts documentation

The codebase contains multiple files that explain the expected flows in the system, along with per-function documentation. The development team provided answers to questions raised by the Nethermind Security team during the sync call.

The code is also accompanied with comprehensive Natspec documentation describing the purpose, inputs, and outputs for each function. Functions with more complex logic also have inline comments to improve code readability.

8 Test Suite Evaluation

8.1 Tests Output

```
[ ] Compiling 40 files with Solc 0.8.20
[ ] Solc 0.8.20 finished in 9.42s
Compiler run successful!

Ran 1 test for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_reclaimTimelockIsAtLeast24Hours() (gas: 8570)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.27ms (63.60µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenAddingOracleAsOperator() (gas: 18151)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.38ms (108.41µs CPU time)

Ran 3 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenAdminCallsCreateVault() (gas: 14428)
[PASS] test_revertsWhenNobodyCallsCreateVault() (gas: 14450)
[PASS] test_revertsWhenOperatorCallsCreateVault() (gas: 14450)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.41ms (339.07µs CPU time)

Ran 3 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenAdminCallsSetMinFirstLiq() (gas: 21951)
[PASS] test_revertsWhenNobodyCallsSetMinFirstLiq() (gas: 21931)
[PASS] test_revertsWhenOperatorCallsSetMinFirstLiq() (gas: 21975)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.57ms (312.12µs CPU time)

Ran 2 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_cumulativeFeeGrowthEqualsSum() (gas: 67482)
[PASS] test_twoEventsEmittedWithCumulativeValues() (gas: 71838)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.96ms (401.36µs CPU time)

Ran 4 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_revertsWhenAdminCalls() (gas: 22500)
[PASS] test_revertsWhenArbitraryAddressCalls() (gas: 22684)
[PASS] test_revertsWhenLpCalls() (gas: 22521)
[PASS] test_revertsWhenOracleCalls() (gas: 22476)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.19ms (391.21µs CPU time)

Ran 1 test for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_inRangePositionGetsCorrectFees() (gas: 130529)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.46ms (312.85µs CPU time)

Ran 1 test for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_revertsOnZeroAmount() (gas: 24941)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.27ms (110.36µs CPU time)

Ran 10 tests for test/features/FEAT-J92H-deploy-contracts/
[PASS] test_factoryAdminIsRegistered() (gas: 10642)
[PASS] test_factoryConditionalTokensMatchesEnvVar() (gas: 8141)
[PASS] test_factoryDeployedAtNonZeroAddress() (gas: 2516)
[PASS] test_factoryExchangeMatchesEnvVar() (gas: 8207)
[PASS] test_factoryImplementationMatchesDeployedImpl() (gas: 10254)
[PASS] test_factoryOperatorIsRegistered() (gas: 10730)
[PASS] test_factoryOracleMatchesEnvVar() (gas: 10166)
[PASS] test_factoryUsdcMatchesEnvVar() (gas: 8076)
[PASS] test_implementationDeployedAtNonZeroAddress() (gas: 2494)
[PASS] test_implementationInitializeReverts() (gas: 22099)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 4.45ms (561.73µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsOnDuplicateMarketId() (gas: 393718)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.49ms (279.97µs CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_emitsMinimumFirstLiquidityUpdatedEvent() (gas: 32990)
[PASS] test_oracleUpdatesMinimumFirstLiquidity() (gas: 29741)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.64ms (205.66µs CPU time)

Ran 4 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_emitsFeesNotifiedEvent() (gas: 61138)
```

```
[PASS] test_feeGrowthGlobalIncreasesByCorrectDelta() (gas: 59755)
[PASS] test_noUsdcTransferDuringNotify() (gas: 65333)
[PASS] test_positionStateUnchanged() (gas: 72377)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.91ms (455.04µs CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_addOperatorEmitsEvent() (gas: 44059)
[PASS] test_addOperatorRegistersAddress() (gas: 43415)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.51ms (687.67µs CPU time)

Ran 1 test for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_revertsBeforeTimelockElapses() (gas: 69460)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.83ms (1.10ms CPU time)

Ran 1 test for test/features/FEAT-J92H-deploy-contracts/
[PASS] test_deploymentProducesSameResultsRegardlessOfVerifyFlag() (gas: 13154227)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.38ms (2.65ms CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsOnZeroNewMin() (gas: 21935)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.47ms (86.94µs CPU time)

Ran 2 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_accumulatedFeesNeverExceedNotifiedAmount() (gas: 56258)
[PASS] test_truncatesDownward() (gas: 56077)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.01ms (202.52µs CPU time)

Ran 3 tests for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_secondCollectPaysOnlyNewFees() (gas: 156824)
[PASS] test_secondEventHasDeltaAmount() (gas: 159091)
[PASS] test_snapshotUpdatedAfterSecondCollect() (gas: 153129)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.47ms (1.59ms CPU time)

Ran 1 test for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_nonReentrantApplied() (gas: 122792)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.53ms (279.53µs CPU time)

Ran 3 tests for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_noTransferWhenZeroFees() (gas: 55822)
[PASS] test_snapshotUnchangedOnZeroCollect() (gas: 48364)
[PASS] test_succeedsWithoutRevert() (gas: 39909)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 10.27ms (363.61µs CPU time)

Ran 1 test for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_revertsForInvalidPositionId() (gas: 23663)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.62ms (69.43µs CPU time)

Ran 6 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_onlyAdminRevertsForNonAdmin() (gas: 21967)
[PASS] test_onlyFactoryPassesForFactory() (gas: 15506)
[PASS] test_onlyFactoryRevertsForNonFactory() (gas: 16079)
[PASS] test_onlyOperatorRevertsForNonOperator() (gas: 22034)
[PASS] test_onlyOraclePassesForOracle() (gas: 21097)
[PASS] test_onlyOracleRevertsForNonOracle() (gas: 21579)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 3.34ms (404.32µs CPU time)

Ran 2 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] test_feeGrowthUnchangedAfterRevert() (gas: 31768)
[PASS] test_revertsWhenNoActiveLiquidity() (gas: 28620)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.16ms (220.24µs CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_vaultAdminsReadsFromFactory() (gas: 26871)
[PASS] test_vaultAdminsReturnsZeroForNonAdmin() (gas: 29282)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.23ms (415.50µs CPU time)

Ran 4 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_addOperatorRevertsForNonAdmin() (gas: 16133)
[PASS] test_removeOperatorRevertsForNonAdmin() (gas: 16046)
[PASS] test_setOracleRevertsForNonAdmin() (gas: 16133)
[PASS] test_transferAdminRevertsForNonAdmin() (gas: 16245)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.45ms (1.19ms CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_vaultOracleReadsFromFactory() (gas: 24286)
```



```

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.02ms (88.48µs CPU time)

Ran 1 test for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_collectedNeverExceedsDistributed() (gas: 126715)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.93ms (216.99µs CPU time)

Ran 4 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_revertsWhenAdminCalls() (gas: 31220)
[PASS] test_revertsWhenLpCallsDirectly() (gas: 29242)
[PASS] test_revertsWhenNobodyCalls() (gas: 31419)
[PASS] test_revertsWhenOracleCalls() (gas: 31218)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.93ms (2.51ms CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsOnZeroMinimumFirstLiquidity() (gas: 14433)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.88ms (62.26µs CPU time)

Ran 1 test for test/features/FEAT-J92H-deploy-contracts/
[PASS] test_deployAcceptsOnlyAddressParams() (gas: 13145081)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.10ms (3.23ms CPU time)

Ran 4 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_revertsOnEmptySignature() (gas: 42665)
[PASS] test_revertsOnHighSValue() (gas: 47683)
[PASS] test_revertsOnInvalidV() (gas: 47501)
[PASS] test_revertsWhenSignerMismatch() (gas: 53468)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.18ms (1.73ms CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_removeOperatorClearsMapping() (gas: 21278)
[PASS] test_removeOperatorEmitsEvent() (gas: 19947)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.79ms (152.28µs CPU time)

Ran 2 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_revertsWhenFirstMintBelowFloor() (gas: 76881)
[PASS] test_succeedsWhenFirstMintMeetsFloor() (gas: 292765)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.95ms (1.30ms CPU time)

Ran 3 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_emergencyCancelRevertsAfterOperatorActivity() (gas: 59929)
[PASS] test_notifyFeesResetsTimestamp() (gas: 51656)
[PASS] test_updateTickResetsTimestamp() (gas: 60899)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.30ms (358.78µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsOnDoubleInitialize() (gas: 399348)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.34ms (750.05µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenOracleEqualsOperator() (gas: 4669643)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.65ms (1.19ms CPU time)

Ran 1 test for test/features/FEAT-J92H-deploy-contracts/
[PASS] test_revertsWhenOracleEqualsOperator() (gas: 11677563)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.35ms (1.70ms CPU time)

Ran 4 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_arbitraryCannotSchedule() (gas: 4632823)
[PASS] test_operatorCannotApply() (gas: 13413)
[PASS] test_operatorCannotCancel() (gas: 13303)
[PASS] test_operatorCannotSchedule() (gas: 4632744)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.78ms (6.91ms CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_vaultReflectsLiveFactoryState() (gas: 57466)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.70ms (310.85µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenSettingOperatorAsOracle() (gas: 18127)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.97ms (106.56µs CPU time)

Ran 2 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_revertsWhenCallerHasNoPosition() (gas: 28127)
[PASS] test_revertsWhenOperatorHasNoPosition() (gas: 27985)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 7.14ms (301.12µs CPU time)

```



```

Ran 9 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_cloneActiveLiquidityIsZero() (gas: 395841)
[PASS] test_cloneConfigMatchesFactoryValues() (gas: 408989)
[PASS] test_cloneMarketIdMatches() (gas: 393830)
[PASS] test_cloneMinimumFirstLiquidityMatches() (gas: 394050)
[PASS] test_clonePhaseIsActive() (gas: 393817)
[PASS] test_conditionalTokensApprovalOnExchange() (gas: 398874)
[PASS] test_emitsVaultCreatedEvent() (gas: 400517)
[PASS] test_usdcApprovalIsMaxOnExchange() (gas: 398810)
[PASS] test_vaultForMarketReturnsCloneAddress() (gas: 394573)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 12.33ms (10.86ms CPU time)

Ran 2 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_feeGrowthOutsidePreservedOnExistingTick() (gas: 285621)
[PASS] test_liquidityGrossAccumulatesOnExistingTick() (gas: 212649)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 7.59ms (2.39ms CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenNonFactoryCallsInitialize() (gas: 86953)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.94ms (862.72µs CPU time)

Ran 8 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_adminCountIsOne() (gas: 7997)
[PASS] test_conditionalTokensAddressIsStored() (gas: 8192)
[PASS] test_exchangeAddressIsStored() (gas: 8258)
[PASS] test_implementationAddressIsStored() (gas: 10283)
[PASS] test_initialAdminIsRegistered() (gas: 10736)
[PASS] test_initialOperatorIsRegistered() (gas: 10695)
[PASS] test_oracleIsSet() (gas: 10261)
[PASS] test_usdcAddressIsStored() (gas: 8105)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 3.74ms (1.31ms CPU time)

Ran 5 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_allPositionsZeroed() (gas: 118074)
[PASS] test_lpAReceivesCorrectTotal() (gas: 105123)
[PASS] test_lpBReceivesCorrectTotal() (gas: 107112)
[PASS] test_phaseCancelled() (gas: 101628)
[PASS] test_vaultBalanceZeroOrDust() (gas: 103492)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 18.74ms (14.53ms CPU time)

Ran 3 tests for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_revertsWhenAdminCalls() (gas: 25948)
[PASS] test_revertsWhenNonOwnerCalls() (gas: 26111)
[PASS] test_revertsWhenOperatorCalls() (gas: 25947)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.70ms (393.78µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_setOracleUpdatesAddress() (gas: 24275)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.91ms (302.48µs CPU time)

Ran 6 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_collectReverts() (gas: 23490)
[PASS] test_emergencyCancelAllRevertsAgain() (gas: 21111)
[PASS] test_mintPositionForReverts() (gas: 38643)
[PASS] test_notifyFeesReverts() (gas: 36386)
[PASS] test_startWindDownReverts() (gas: 23699)
[PASS] test_updateTickReverts() (gas: 29913)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 4.74ms (824.93µs CPU time)

Ran 4 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_emitsAppliedEvent() (gas: 4656557)
[PASS] test_implementationUpdated() (gas: 4655216)
[PASS] test_pendingCleared() (gas: 4656582)
[PASS] test_versionIncremented() (gas: 4656288)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.78ms (1.58ms CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_vaultOperatorsReadsFromFactory() (gas: 26901)
[PASS] test_vaultOperatorsReturnsZeroForNonOperator() (gas: 29312)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.38ms (405.86µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenInitializeCalledOnImplementation() (gas: 4632541)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.15ms (1.30ms CPU time)

```

```

Ran 6 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_onlyAdminPassesForAdmin() (gas: 12865)
[PASS] test_onlyAdminRevertsForNonAdmin() (gas: 13391)
[PASS] test_onlyOperatorPassesForOperator() (gas: 12798)
[PASS] test_onlyOperatorRevertsForNonOperator() (gas: 13324)
[PASS] test_onlyOraclePassesForOracle() (gas: 12843)
[PASS] test_onlyOracleRevertsForNonOracle() (gas: 13304)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.68ms (253.87µs CPU time)

Ran 6 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_activeLiquidityZeroed() (gas: 68325)
[PASS] test_emitsEmergencyCancelExecutedEvent() (gas: 67790)
[PASS] test_lpReceivesPrincipalPlusFees() (gas: 71441)
[PASS] test_phaseChangesToCancelled() (gas: 66896)
[PASS] test_positionLiquidityZeroed() (gas: 71683)
[PASS] test_vaultBalanceZeroOrDust() (gas: 69259)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 4.79ms (1.05ms CPU time)

Ran 4 tests for test/features/FEAT-JGE7-vault-wind-down-lifecycle/
[PASS] test_revertsWhenAdminCalls() (gas: 21506)
[PASS] test_revertsWhenArbitraryAddressCalls() (gas: 21690)
[PASS] test_revertsWhenLpCalls() (gas: 21527)
[PASS] test_revertsWhenOperatorCalls() (gas: 21505)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.87ms (498.60µs CPU time)

Ran 2 tests for test/features/FEAT-JXQ0-emergency-cancel-all-positions/
[PASS] test_noStateChangeOnRevert() (gas: 31184)
[PASS] test_revertsBeforeTimelockElapsed() (gas: 23378)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 7.90ms (162.92µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenProposingExistingAdmin() (gas: 14271)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.32ms (60.64µs CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenCallerIsNotPendingAdmin() (gas: 13285)
[PASS] test_revertsWhenWrongCallerAfterTransferProposed() (gas: 45686)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.55ms (313.44µs CPU time)

Ran 3 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_emitsCancelledEvent() (gas: 4644375)
[PASS] test_implementationUnchanged() (gas: 4646887)
[PASS] test_pendingCleared() (gas: 4644881)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.70ms (915.11µs CPU time)

Ran 1 test for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_revertsBeforeTimelock() (gas: 4681718)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.72ms (777.95µs CPU time)

Ran 1 test for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_revertsOnDuplicateIntentId() (gas: 310538)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.48ms (1.11ms CPU time)

Ran 3 tests for test/features/FEAT-K1MD-pause-trading/
[PASS] test_emitsTradingUnpausedEvent() (gas: 32021)
[PASS] test_notifyFeesSucceedsAfterUnpause() (gas: 75676)
[PASS] test_pausedFlagSetToFalse() (gas: 32717)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.36ms (717.62µs CPU time)

Ran 1 test for test/features/FEAT-JGE7-vault-wind-down-lifecycle/
[PASS] test_revertsOnSecondCall() (gas: 23720)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.29ms (72.19µs CPU time)

Ran 1 test for test/features/FEAT-K1MD-pause-trading/
[PASS] test_collectSucceedsWhilePaused() (gas: 138280)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.81ms (264.64µs CPU time)

Ran 1 test for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_revertsWhenProposingZeroAddress() (gas: 13945)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.84ms (55.58µs CPU time)

Ran 3 tests for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_collectSucceedsInWindDown() (gas: 85173)
[PASS] test_emitsEventInWindDown() (gas: 86590)

```

```
[PASS] test_phaseUnchangedAfterCollect() (gas: 80982)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 21.77ms (661.68µs CPU time)

Ran 2 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_cloneCanBeInitialized() (gas: 6655469)
[PASS] test_cloneCannotBeInitializedTwice() (gas: 6866295)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 12.89ms (14.38ms CPU time)

Ran 1 test for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_revertsWhenVaultNotActive() (gas: 31402)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.82ms (108.85µs CPU time)

Ran 3 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_activeLiquidityUnchangedWhenOutOfRange() (gas: 280355)
[PASS] test_bothTicksInitializedWithZeroFeeGrowthOutside() (gas: 283657)
[PASS] test_positionCreatedAndUsdcTransferred() (gas: 287942)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.88ms (2.47ms CPU time)

Ran 2 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_newVaultHasUpdatedVersion() (gas: 5309138)
[PASS] test_oldVaultVersionUnchanged() (gas: 4988390)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.68ms (1.21ms CPU time)

Ran 3 tests for test/features/FEAT-K1M2-merge-positions/
[PASS] test_collectAfterMergeReturnsCorrectTotal() (gas: 129244)
[PASS] test_survivorFeeGrowthInsideLastEqualsCurrentValue() (gas: 109707)
[PASS] test_survivorTokensOwedIncludesBothFees() (gas: 120091)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.16ms (962.75µs CPU time)

Ran 3 tests for test/features/FEAT-JGE7-vault-wind-down-lifecycle/
[PASS] test_emitsVaultWindDownStartedEvent() (gas: 33893)
[PASS] test_noSideEffectsOnPositionState() (gas: 63034)
[PASS] test_phaseChangesToWindDown() (gas: 32605)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.31ms (387.30µs CPU time)

Ran 6 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_revertsOnEqualTicks() (gas: 38716)
[PASS] test_revertsOnInvertedRange() (gas: 38760)
[PASS] test_revertsOnMisalignedLowerTick() (gas: 41174)
[PASS] test_revertsOnMisalignedUpperTick() (gas: 41555)
[PASS] test_revertsOnZeroAmount() (gas: 38719)
[PASS] test_revertsWhenVaultNotActive() (gas: 36089)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 16.06ms (13.72ms CPU time)

Ran 4 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_activeLiquidityUnchanged() (gas: 61593)
[PASS] test_currentTickUpdated() (gas: 58118)
[PASS] test_emitsTickUpdatedWithZeroCrossings() (gas: 59915)
[PASS] test_lastOperatorActivityTimestampUpdated() (gas: 61506)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 5.26ms (458.31µs CPU time)

Ran 3 tests for test/features/FEAT-K1M2-merge-positions/
[PASS] test_revertsWhenAdminCalls() (gas: 27807)
[PASS] test_revertsWhenArbitraryAddressCalls() (gas: 27960)
[PASS] test_revertsWhenLpCalls() (gas: 27828)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.94ms (448.68µs CPU time)

Ran 2 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_applyRevertsNoPending() (gas: 15516)
[PASS] test_cancelRevertsNoPending() (gas: 15470)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.82ms (120.23µs CPU time)

Ran 1 test for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_revertsWhenIntentAlreadyFulfilled() (gas: 66854)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.43ms (1.09ms CPU time)

Ran 4 tests for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_phase1EmitsReclaimSubmitted() (gas: 91353)
[PASS] test_phase1NoUsdcTransferred() (gas: 103886)
[PASS] test_phase1RecordsTimestamp() (gas: 89971)
[PASS] test_phase1UsedIntentsRemainsFalse() (gas: 89915)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.93ms (6.30ms CPU time)

Ran 1 test for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_reentrancyDuringPhase2TransferIsBlocked() (gas: 558381)
```

```

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.88ms (1.62ms CPU time)

Ran 4 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_revertsWhenAdminCalls() (gas: 22434)
[PASS] test_revertsWhenArbitraryAddressCalls() (gas: 22606)
[PASS] test_revertsWhenLpCalls() (gas: 22455)
[PASS] test_revertsWhenOracleCalls() (gas: 22410)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.08ms (484.33µs CPU time)

Ran 2 tests for test/features/FEAT-K1M2-merge-positions/
[PASS] test_revertsOnEmptyArray() (gas: 30101)
[PASS] test_revertsOnSingleElement() (gas: 32594)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.76ms (229.52µs CPU time)

Ran 6 tests for test/features/FEAT-K1MD-pause-trading/
[PASS] test_emitsTradingPausedEvent() (gas: 30148)
[PASS] test_mergePositionsRevertsWhilePaused() (gas: 201222)
[PASS] test_mintPositionForRevertsWhilePaused() (gas: 44965)
[PASS] test_notifyFeesRevertsWhilePaused() (gas: 36268)
[PASS] test_pausedFlagSetToTrue() (gas: 30881)
[PASS] test_updateTickRevertsWhilePaused() (gas: 36158)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 6.17ms (1.68ms CPU time)

Ran 7 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_acceptAdminClearsPendingAdmin() (gas: 53074)
[PASS] test_acceptAdminEmitsEvent() (gas: 53995)
[PASS] test_acceptAdminGrantsRole() (gas: 53511)
[PASS] test_acceptAdminIncrementsAdminCount() (gas: 54080)
[PASS] test_transferAdminDoesNotGrantRole() (gas: 43472)
[PASS] test_transferAdminEmitsEvent() (gas: 44081)
[PASS] test_transferAdminSetsPendingAdmin() (gas: 42998)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 7.68ms (5.19ms CPU time)

Ran 3 tests for test/features/FEAT-K1MD-pause-trading/
[PASS] test_arbitraryCannotUnpause() (gas: 22101)
[PASS] test_lpCannotPause() (gas: 21984)
[PASS] test_operatorCannotPause() (gas: 21939)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.38ms (388.23µs CPU time)

Ran 6 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_bitmapCrossWordBoundary() (gas: 238111)
[PASS] test_bitmapSetAtTick0() (gas: 11134)
[PASS] test_bitmapSetAtTick100() (gas: 11113)
[PASS] test_bitmapSetAtTick200() (gas: 11135)
[PASS] test_uninitializedTickNotInBitmap() (gas: 11110)
[PASS] test_updateTickCrossesWordBoundary() (gas: 350725)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 9.11ms (3.82ms CPU time)

Ran 5 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_activeLiquidityAdjusted() (gas: 93611)
[PASS] test_currentTickUpdated() (gas: 92079)
[PASS] test_emitsTickUpdatedEvent() (gas: 93810)
[PASS] test_feeGrowthOutsideFlipped() (gas: 97799)
[PASS] test_lastOperatorActivityTimestampUpdated() (gas: 95444)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.46ms (788.95µs CPU time)

Ran 1 test for test/features/FEAT-K1MD-pause-trading/
[PASS] test_phaseUnchangedAfterPauseUnpause() (gas: 34080)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.20ms (191.59µs CPU time)

Ran 6 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_activeLiquidityAdjusted() (gas: 54971)
[PASS] test_currentTickUpdated() (gas: 53451)
[PASS] test_emitsTickUpdatedEvent() (gas: 55205)
[PASS] test_feeGrowthOutsideFlipUsesOldValue() (gas: 75476)
[PASS] test_feeGrowthOutsideFlippedBack() (gas: 59121)
[PASS] test_lastOperatorActivityTimestampUpdated() (gas: 56838)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 6.12ms (1.15ms CPU time)

Ran 3 tests for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_revertsOnHighSOperatorSignature() (gas: 56461)
[PASS] test_revertsOnInvalidVOperatorSignature() (gas: 56290)
[PASS] test_revertsWhenSignerIsNotOperator() (gas: 68242)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.33ms (2.11ms CPU time)

```



```
Ran 5 tests for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_emitsScheduledEvent() (gas: 4681322)
[PASS] test_implementationUnchanged() (gas: 4683254)
[PASS] test_pendingImplementationSet() (gas: 4679961)
[PASS] test_unlockAtSet() (gas: 4679985)
[PASS] test_versionUnchanged() (gas: 4682760)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.92ms (3.53ms CPU time)

Ran 4 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_fullOracleRotationVerifiesBothDirections() (gas: 53389)
[PASS] test_newOracleAcceptedAfterRotation() (gas: 43155)
[PASS] test_newOracleEmitsEventOnVault() (gas: 44379)
[PASS] test_oldOracleRejectedAfterRotation() (gas: 37400)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.05ms (472.27µs CPU time)

Ran 3 tests for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_phase2EmitsDepositReclaimed() (gas: 129079)
[PASS] test_phase2MarksUsedIntent() (gas: 127769)
[PASS] test_phase2TransfersUsdcToLp() (gas: 135811)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.55ms (2.59ms CPU time)

Ran 4 tests for test/features/FEAT-JGE7-vault-wind-down-lifecycle/
[PASS] test_collectEmitsEventInWindDown() (gas: 93056)
[PASS] test_collectSucceedsInWindDown() (gas: 85195)
[PASS] test_phaseUnchangedAfterCollect() (gas: 81004)
[PASS] test_tokensOwnedZeroedAfterCollectInWindDown() (gas: 83761)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.96ms (972.81µs CPU time)

Ran 4 tests for test/features/FEAT-REPZ-deploy-lp-vault-for-a-market/
[PASS] test_fullRotationCycleVerifiesBothDirections() (gas: 73798)
[PASS] test_newOperatorAcceptedAfterRotation() (gas: 59268)
[PASS] test_oldOperatorRejectedAfterRotation() (gas: 30624)
[PASS] test_rotationEmitsFactoryEvents() (gas: 53074)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.07ms (646.08µs CPU time)

Ran 1 test for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_revertsOnDoubleSchedule() (gas: 9296971)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 15.40ms (12.66ms CPU time)

Ran 1 test for test/features/FEAT-KX5N-upgradeable-vault-impl-pointer/
[PASS] test_revertsOnZeroAddress() (gas: 13878)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.53ms (61.31µs CPU time)

Ran 4 tests for test/features/FEAT-K1M2-merge-positions/
[PASS] test_consumedPositionLiquidityIsZero() (gas: 70597)
[PASS] test_emitsPositionsMergedEvent() (gas: 70571)
[PASS] test_survivorLiquidityEqualsSumOfBoth() (gas: 79412)
[PASS] test_tickLiquidityGrossUnchanged() (gas: 81730)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.74ms (955.55µs CPU time)

Ran 3 tests for test/features/FEAT-JGE7-vault-wind-down-lifecycle/
[PASS] test_intentIdNotConsumedAfterRevert() (gas: 45998)
[PASS] test_mintPositionForRevertsInWindDown() (gas: 38687)
[PASS] test_nextPositionIdUnchangedAfterRevert() (gas: 44433)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.98ms (1.53ms CPU time)

Ran 1 test for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_revertsOnSameTick() (gas: 33405)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.85ms (110.41µs CPU time)

Ran 2 tests for test/features/FEAT-K1M2-merge-positions/
[PASS] test_noStateChangeOnMismatch() (gas: 80326)
[PASS] test_revertsOnRangeMismatch() (gas: 57218)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 13.55ms (441.87µs CPU time)

Ran 1 test for test/features/FEAT-K1MD-pause-trading/
[PASS] test_reclaimDepositSucceedsWhilePaused() (gas: 157372)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.55ms (1.97ms CPU time)

Ran 1 test for test/features/FEAT-JAIJ-lp-escape-hatch/
[PASS] test_revertsOnReplayAfterSuccessfulReclaim() (gas: 66875)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 9.85ms (1.06ms CPU time)

Ran 5 tests for test/features/FEAT-U079-collect-fees-on-a-position/
[PASS] test_emitsFeesCollectedEvent() (gas: 86611)
```



```
[PASS] test_lpReceivesOwedUsdc() (gas: 89227)
[PASS] test_positionLiquidityUnchanged() (gas: 88401)
[PASS] test_snapshotUpdatedAfterCollect() (gas: 85059)
[PASS] test_vaultBalanceDecreasesByOwed() (gas: 89293)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 29.13ms (1.31ms CPU time)

Ran 2 tests for test/Counter.t.sol:CounterTest
[PASS] testFuzz_SetNumber(uint256) (runs: 256, : 29133, ~: 29289)
[PASS] test_Increment() (gas: 28784)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 13.09ms (12.90ms CPU time)

Ran 3 tests for test/features/FEAT-TVS0-update-tick-and-cross-ticks/
[PASS] test_revertsRightToLeftTooManyTicks() (gas: 3188293)
[PASS] test_revertsTooManyTicksCrossed() (gas: 2232058)
[PASS] test_stateUnchangedAfterRevert() (gas: 2234838)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 86.15ms (27.01ms CPU time)

Ran 6 tests for test/features/FEAT-J92H-deploy-contracts/
[PASS] test_revertsWhenAdminIsZero() (gas: 23205)
[PASS] test_revertsWhenConditionalTokensIsZero() (gas: 23170)
[PASS] test_revertsWhenExchangeIsZero() (gas: 23136)
[PASS] test_revertsWhenOperatorIsZero() (gas: 23294)
[PASS] test_revertsWhenOracleIsZero() (gas: 23215)
[PASS] test_revertsWhenUsdcIsZero() (gas: 23123)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 111.26ms (435.70µs CPU time)

Ran 9 tests for test/features/FEAT-T7AF-mint-lp-position/
[PASS] test_activeLiquidityIncreasedForInRangePosition() (gas: 321255)
[PASS] test_emitsPositionMintedEvent() (gas: 323311)
[PASS] test_feeGrowthInsideSnapshotPreventsRetroactiveClaims() (gas: 322360)
[PASS] test_intentIdRecordedAsUsed() (gas: 320119)
[PASS] test_lowerTickInitializedCorrectly() (gas: 321597)
[PASS] test_nextPositionIdIncremented() (gas: 321224)
[PASS] test_positionRecordIsCorrect() (gas: 323440)
[PASS] test_upperTickInitializedCorrectly() (gas: 321682)
[PASS] test_usdcTransferredFromLpToVault() (gas: 328171)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 104.34ms (8.24ms CPU time)

Ran 2 tests for test/features/FEAT-TOGR-notify-and-distribute-fees/
[PASS] testFuzz_largeAmountDoesNotOverflow(uint256) (runs: 256, : 59013, ~: 59229)
[PASS] test_revertsWhenMulDivResultOverflows() (gas: 32990)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 147.06ms (144.77ms CPU time)

Ran 108 test suites in 154.28ms (1.02s CPU time): 308 tests passed, 0 failed, 0 skipped (308 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.